CMSC132: Hashing

By Naren Ayinala

May 2026

Contents

1 What Is Hashing? 2

2 The equals/hashCode Contract 2
2.1 Why a constant hash code is legal but bad 2
2.2 Correct custom equality 2

3 What Is a Collision? 3

4 Separate Chaining 3

5 Open Addressing: All Entries in the Array 3
5.1 Linear probing 3
5.2 Quadratic probing 3
5.3 Double hashing 4

6 The Three Slot States and Tombstones 4

7 Load Factor and Rehashing 4
7.1 Why resizing costs $O(n)$ but average insertion can remain $O(1)$ 4

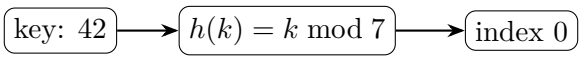
8 Implementing a Small Hash Table in Java 4

9 Complexity and Common Traps 5

10 Practice Questions with Answers 6

1 What Is Hashing?

Suppose you want to store student records by ID. Scanning every student would take $O(n)$ time. A **hash table** uses a **hash function** to turn a key into a position in an array, allowing expected $O(1)$ lookup, insertion, and removal when collisions are handled well and the load factor stays controlled.



Here $42 \bmod 7 = 0$. The modulus example is intentionally simple; production hash tables usually mix hash bits before choosing a bucket. For a negative Java `hashCode()`, naive `hash \% capacity` may be negative, so the index calculation must account for that.

2 The equals/hashCode Contract

Every Java object inherits `equals` and `hashCode` from `Object`. Contrary to a common misconception, a hash code is *not* guaranteed to be unique to an object. Two different objects can have the same hash code.

1. If `a.equals(b)` is true, `a.hashCode() == b.hashCode()` must be true.
2. If two hash codes match, the objects are *not necessarily* equal.
3. During an execution, repeated calls to `hashCode()` should be consistent as long as the information used in equality has not changed.

2.1 Why a constant hash code is legal but bad

```
@Override public int hashCode() { return 1; }
```

This satisfies the first rule for all equal objects, but every key falls into the same initial bucket. Collisions then degrade performance. A valid hash function should ideally distribute keys evenly.

2.2 Correct custom equality

Suppose equality depends on a student's ID and campus, but not on their display name. Hashing must use the same equality-relevant information.

```
import java.util.Objects;
class StudentKey {
    final int id;
    final String campus;
    final String displayName;
    StudentKey(int id, String campus, String name) {
        this.id = id; this.campus = campus;
        this.displayName = name;
    }
    @Override public boolean equals(Object other) {
        if (this == other) return true;
        if (!(other instanceof StudentKey)) return false;
        StudentKey s = (StudentKey) other;
        return id == s.id && Objects.equals(campus, s.campus);
    }
    @Override public int hashCode() {
        return Objects.hash(id, campus);
    }
}
```

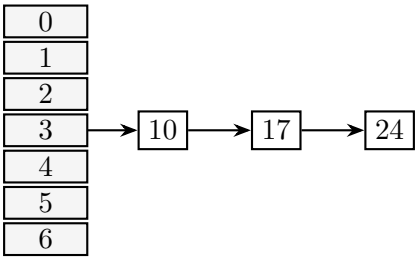
If you change equality-relevant fields after placing an object into a hash set or map, later lookups may fail. Prefer immutable keys.

3 What Is a Collision?

A **collision** occurs when distinct keys map to the same table index. Even perfect-looking hash functions cannot prevent all collisions when there are more possible keys than buckets. Two main collision-handling strategies are **separate chaining** and **open addressing**.

4 Separate Chaining

Each array slot stores a **bucket** capable of holding multiple entries, often as a linked list. If 10, 17, and 24 are inserted into a seven-bucket table with $h(k) = k \bmod 7$, all three go into bucket 3.



To search for 17, compute bucket 3, then compare the entries in that bucket using equality until you find 17. With well-distributed keys and controlled load, chains stay short on average. In the worst case, all keys collide and a basic linked-list chain can require $O(n)$ comparisons.

5 Open Addressing: All Entries in the Array

Instead of storing a chain at one index, open addressing probes other slots until it finds an available location. A key rule is that **search must follow the same probe sequence used during insertion**.

5.1 Linear probing

If the original index is $h(k)$, probe

$$h(k),\ (h(k)+1) \bmod m,\ (h(k)+2) \bmod m, \ldots$$

For a table of length 7, inserting 10, 17, and 24 gives the following layout, because all three initially hash to 3:

–	–	–	10	17	24	–
0	1	2	3	4	5	6

Linear probing is simple and cache-friendly, but can suffer **primary clustering**: contiguous occupied runs become increasingly long.

5.2 Quadratic probing

Probe indices can follow $h(k) + i^2 \pmod m$ for $i = 0, 1, 2, \ldots$. For a starting index of 3 in a table of size 7, the probes are 3, 4, 0, 5, ... Unlike linear probing, not every table configuration guarantees that every slot is visited; capacity choice and load constraints matter. Quadratic probing reduces primary clustering but keys with the same initial index still follow the same probe sequence (**secondary clustering**).

5.3 Double hashing

Use a second hash function for the step size:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m.$$

For example, with $m = 7$, $h_1(k) = k \bmod 7$, and $h_2(k) = 1 + (k \bmod 6)$, the key 10 starts at 3 with step 5. Its probes are 3, 1, 6, 4, 2, 0, 5. The step must be nonzero and relatively prime to table capacity to guarantee visiting all slots.

6 The Three Slot States and Tombstones

In open addressing, a slot can be **NEVER USED**, **OCCUPIED**, or **REMOVED**. A removed slot is also called a **tombstone**. Why not just set a removed slot back to NEVER USED? Because a later key might have been placed beyond that slot during a collision.

empty	empty	empty	10	R	24	empty
0	1	2	3	4	5	6

If we deleted 17 at index 4, searching for 24 must *continue* past index 4. Stopping there would incorrectly report that 24 is absent. However, encountering a NEVER USED slot during a correctly implemented probe search means the key was not inserted further along that sequence. During insertion, a tombstone can usually be reused, but searching for an existing key must still inspect subsequent positions.

7 Load Factor and Rehashing

The **load factor** is

$$\lambda = \frac{\text{number of stored entries}}{\text{number of buckets or slots}}.$$

For example, 6 entries in 8 slots gives $\lambda = 6/8 = 0.75$. Higher load factors usually increase collisions. When a chosen threshold is exceeded, the table may allocate a larger array and **rehash** existing entries into their new positions.

Remember: The new capacity is *implementation-dependent*. It is not a universal rule that every Java hash table doubles its size and rounds to the nearest prime. The original notes describe one possible classroom resizing strategy, not a Java-wide guarantee.

7.1 Why resizing costs $O(n)$ but average insertion can remain $O(1)$

Rehashing a table with n entries requires visiting and reinserting its contents, so a particular insertion may take $O(n)$. But if growth happens only occasionally, that cost is spread over many insertions. Under standard assumptions, the **amortized** insertion cost remains $O(1)$.

8 Implementing a Small Hash Table in Java

This example uses separate chaining and integer keys. It is deliberately educational: the real Java `HashMap` is more sophisticated. We resize when the load factor exceeds 0.75.

```
import java.util.*;
public class IntHashSet {
    private List<Integer>[] buckets;
    private int size;
```

```
    @SuppressWarnings("unchecked")
    public IntHashSet() {
        buckets = (List<Integer>[]) new List[8];
        for (int i = 0; i < buckets.length; i++)
            buckets[i] = new LinkedList<>();
    }
    private int index(int key) {
        return Math.floorMod(Integer.hashCode(key), buckets.length);
    }
    public boolean contains(int key) {
        return buckets[index(key)].contains(key);
    }
    public boolean add(int key) {
        if (contains(key)) return false;
        buckets[index(key)].add(key);
        size++;
        if ((double) size / buckets.length > 0.75) resize();
        return true;
    }
    @SuppressWarnings("unchecked")
    private void resize() {
        List<Integer>[] old = buckets;
        buckets = (List<Integer>[]) new List[old.length * 2];
        for (int i = 0; i < buckets.length; i++)
            buckets[i] = new LinkedList<>();
        for (List<Integer> chain : old)
            for (int key : chain) buckets[index(key)].add(key);
    }
    public int size() { return size; }
}
```

The important details are (1) `floorMod` avoids negative indices, (2) duplicates are rejected, and (3) entries must be redistributed when the array length changes. Simply copying each old bucket into the same index would be incorrect.

9 Complexity and Common Traps

Operation	Expected / average	Worst case in a simple implementation
Insert	$O(1)$ amortized	$O(n)$
Search	$O(1)$	$O(n)$
Remove	$O(1)$	$O(n)$
Rehash entire table	$O(n)$	$O(n)$

1. Equal hash codes do not prove equal objects.
2. The hash index depends on capacity, not just the key.
3. In open addressing, REMOVED and NEVER USED are different states.
4. For double hashing, the step size must permit full coverage of the table.
5. Changing a key after insertion can break lookups.
6. $O(1)$ hash-table operations are expected or amortized claims, not unconditional worst-case guarantees.

10 Practice Questions with Answers

Q1. Insert 9, 16, 23 into a seven-slot table using linear probing and $h(k) = k \bmod 7$. **Answer:** indices 2, 3, 4.

Q2. Why can't deletion simply mark a slot as NEVER USED? **Answer:** It may prematurely stop searches for keys displaced farther along the same probe sequence.

Q3. Is returning 1 for every `hashCode` legal? **Answer:** Yes for the equality requirement, but collisions make it inefficient.

Q4. What is the load factor for 12 entries in 16 slots? **Answer:** $12/16 = 0.75$.

Q5. What is the key difference between chaining and open addressing? **Answer:** Chaining stores multiple entries in a bucket; open addressing probes other array slots.